LES SYSTÈMES D'EXPLOITATION MULTIPROCESSEURS

Fondements physiques, architecture du noyau, ordonnancement et mécanismes de concurrence

Document de cours complet — Support académique et technique

Introduction : De l'impasse physique à la révolution parallèle

Pendant près de quarante ans, l'informatique a bénéficié d'une dynamique exceptionnelle. Sans modifier la moindre ligne de code, un programme informatique voyait ses exécutions s'accélérer presque automatiquement au fil des ans. Il suffisait d'attendre la génération suivante de processeurs pour doubler la vitesse de calcul. Cette époque dorée reposait sur deux piliers indissociables : la loi de Moore et l'effet de Dennard.

1. L'âge d'or du monocœur

En 1965, Gordon Moore, cofondateur d'Intel, émet une observation empirique : le nombre de transistors intégrés sur une puce en silicium double approximativement tous les deux ans (intervalle réajusté à 18-24 mois en 1975). En soi, rajouter des transistors permet de complexifier le processeur, mais cela ne garantit pas directement une hausse de fréquence. C'est ici qu'intervient la découverte faite par Robert Dennard en 1974.

Dennard démontre qu'en réduisant la taille physique d'un transistor, sa consommation électrique baisse dans les mêmes proportions. En clair : la densité de puissance consommée par unité de surface de silicium reste constante. Les ingénieurs pouvaient donc faire grimper la fréquence d'horloge de la puce (passant de quelques mégahertz à plusieurs gigahertz) sans que la température globale n'atteigne des seuils critiques. Herb Sutter a qualifié cette période d'âge d'or du *"free lunch"* (le repas gratuit) : l'optimisation logicielle découlait directement des progrès du matériel.

Une précaution conceptuelle s'impose : la loi de Moore n'est pas une loi physique, mais un modèle d'observation industrielle. Et contrairement à une idée reçue, elle ne s'est pas arrêtée net au début des années 2000. On continue aujourd'hui de graver davantage de transistors sur une puce. Ce qui s'est effondré vers 2004-2005, c'est l'échelle de Dennard.

> **ANALOGIE : LE VÉHICULE ET L'AUTOROUTE**
> L'âge d'or du monocœur s'apparente à l'amélioration progressive d'une seule voiture sur une autoroute à une voie. Chaque année, les constructeurs doublaient le régime du moteur. Le conducteur n'avait pas à revoir sa façon de conduire : le simple fait d'installer un moteur plus rapide réduisait instantanément la durée du trajet.

2. La rupture physique : les trois murs

Vers 2005, la miniaturisation des transistors a atteint des dimensions atomiques. À cette échelle, les effets quantiques et les courants de fuite ont brisé le modèle de Dennard, heurtant simultanément trois obstacles majeurs :

- **Le mur de la puissance (Power Wall) :** Les transistors miniaturisés laissent fuiter de l'électricité même au repos. La consommation ne baisse plus avec la taille, et la densité thermique sur la puce devient comparables à celle d'une plaque de cuisson ou d'un réacteur nucléaire. Dissiper cette chaleur avec des moyens traditionnels devient impossible.
- **Le mur de la fréquence (Frequency Wall) :** Directement lié à la surchauffe, le rythme d'horloge s'est bloqué autour de 3 à 4 GHz. Pousser la fréquence au-delà demande une tension électrique excessive qui détruirait le composant.
- **Le mur de la mémoire (Memory Wall) :** La vitesse d'exécution du processeur a augmenté bien plus vite que la rapidité de la mémoire RAM. Le processeur passe une part considérable de ses cycles d'horloge inactif, à attendre le transfert des données depuis la mémoire principale.

3. Le paradoxe moderne

Au moment même où la fréquence individuelle des processeurs atteignait son plafond physique, la demande mondiale en capacité de calcul connaissait une explosion sans précédent : simulations scientifiques complexes, traitement massif de données, intelligence artificielle et traitement vidéo en temps réel. Le silicium ne pouvait plus offrir de puissance supplémentaire par le biais d'un cœur unique.

4. La transition fondamentale

L'industrie des semi-conducteurs a donc opéré un virage stratégique majeur : au lieu de concevoir un processeur monolithique plus rapide, les fabricants ont assemblé **plusieurs cœurs de calcul** sur le même morceau de silicium. Le problème ne résidait plus dans la vitesse pure de traitement, mais dans le parallélisme d'exécution.

C'est ici que la charge bascule sur le logiciel, et plus particulièrement sur le système d'exploitation. Un système d'exploitation conçu pour un processeur unique est fondamentalement aveugle face à cette nouvelle architecture. Sans une refonte complète de sa structure, il ne peut utiliser qu'un seul cœur, laissant les autres inutilisés. L'OS doit désormais agir comme un chef d'orchestre capable de répartir les tâches, de réguler l'accès simultané aux ressources partagées et d'empêcher les collisions de données.

> **ANALOGIE : DE LA VOITURE RAPIDE AU CONVOI DE CAMIONS**
>
> Ne pouvant plus construire une voiture roulant à 1000 km/h à cause de la friction et de la surchauffe du moteur, les ingénieurs ont décidé d'envoyer dix camions roulant chacun à 100 km/h. La quantité totale de marchandise transportée par heure est plus grande, mais la gestion de la circulation devient infiniment plus complexe.

5. Définition du système d'exploitation multiprocesseur

Un **système d'exploitation multiprocesseur** est un logiciel système conçu pour gérer, coordonner et exploiter simultanément plusieurs unités de traitement (processeurs physiques ou cœurs intégrés) partageant tout ou partie de la mémoire vive. Son rôle principal consiste à répartir équitablement les processus sur les cœurs disponibles, à synchroniser l'accès aux structures de données communes du système et à fournir aux applications une vision cohérente et protégée du matériel.

Pour le système d'exploitation moderne, la frontière entre plusieurs puces physiques distinctes et plusieurs cœurs gravés sur une même puce est étroite : chaque cœur est perçu comme un processeur logique capable d'exécuter un fil d'instructions de manière autonome.

> *Ce changement d'échelle pose d'emblée une question fondamentale : comment s'organise le système d'exploitation lui-même lorsqu'il doit s'exécuter sur une machine à plusieurs processeurs ? C'est ce que nous allons analyser dans la première partie.*

Partie 1 : Les modèles d'OS et l'organisation du noyau

L'intégration du multiprocessage ne se limite pas à faire exécuter des programmes utilisateur sur plusieurs cœurs. Le système d'exploitation lui-même — et plus spécifiquement son cœur, le noyau — doit tourner sur ces processeurs. La question clé devient alors : *qui a le droit d'exécuter le code du noyau, et comment empêche-t-on deux processeurs de corrompre l'état interne de l'OS ?*

1.1 Rappels et définitions

Le **noyau (kernel)** est la partie centrale et privilégiée du système d'exploitation. Il s'exécute dans un mode matériel protégé (mode kernel) et détient le contrôle direct sur la mémoire, les périphériques d'entrée-sortie et la gestion des processus. Lorsque votre application souhaite lire un fichier ou envoyer une paquet réseau, elle effectue un appel système (system call) qui passe le relais au noyau.

Il est crucial de distinguer la structure matérielle du modèle logiciel :

- **Architecture matérielle :** définit la disposition physique des puces, des bus et de la mémoire.
- **Modèle d'OS :** définit la manière dont le logiciel tire parti de cette architecture. On classe historiquement ces modèles en deux grandes familles : asymétrique (AMP) et symétrique (SMP).

1.2 Le modèle asymétrique (AMP : Asymmetric Multiprocessing)

Dans un système d'exploitation asymétrique, les processeurs n'occupent pas les mêmes fonctions. Un processeur désigné comme le **maître (Master)** est le seul habilité à exécuter le noyau du système d'exploitation. Les autres processeurs, dits **esclaves (Slaves)**, exécutent uniquement le code des applications utilisateur.

Lorsqu'un processus tournant sur un processeur esclave demande une opération d'entrée-sortie ou un appel système, l'esclave ne peut pas traiter la demande lui-même. Il doit interrompre le processeur maître et attendre que celui-ci traite la requête pour son compte.

> **ANALOGIE : LE RESTAURANT AVEC UN SEUL CHEF DE CUISINE**
> L'AMP ressemble à une cuisine de restaurant où plusieurs commis (les processeurs esclaves) découpent les légumes et préparent les assiettes, mais où un unique chef (le processeur maître) détient les clés de la réserve et prépare toutes les sauces. Si trois commis ont besoin d'épices en même temps, ils doivent faire la queue devant le chef.

Avantages de l'AMP : La conception du noyau est extrêmement simple. Comme un seul processeur s'exécute dans le noyau, il n'existe aucun risque de concurrence sur les structures de données internes du système. Il n'y a pas besoin de verrous complexes.

Limites de l'AMP : Le maître constitue un véritable goulot d'étranglement. Si les applications effectuent beaucoup d'appels système, le maître sature pendant que les esclaves restent inactifs. De plus,

c'est un point individuel de défaillance (SPOF : Single Point of Failure) : si le processeur maître tombe en panne, l'intégralité de la machine s'arrête.

Nuance terminologique : Le terme "asymétrique" est également employé aujourd'hui pour désigner du matériel hétérogène (comme les puces ARM big.LITTLE). Il convient de ne pas confondre l'asymétrie matérielle (cœurs de tailles différentes) et l'asymétrie logicielle de l'OS (répartition maître/esclave des fonctions du noyau).

1.3 Le modèle symétrique (SMP : Symmetric Multiprocessing)

Dans le modèle symétrique, tous les processeurs sont égaux. Chacun d'eux possède les mêmes droits d'accès au matériel et à la mémoire partagée, et surtout, **chaque processeur peut exécuter le code du noyau**. Si un processus effectue un appel système sur le cœur n°3, c'est le cœur n°3 qui bascule en mode noyau et traite la demande sur place, sans transférer le travail à un tiers.

C'est le modèle retenu par la quasi-totalité des systèmes d'exploitation modernes grand public et serveur (Linux, Windows, macOS, FreeBSD).

Avantages du SMP : La charge de travail est répartie de façon homogène sur tous les cœurs disponibles. Si un processeur tombe en panne, le système peut théoriquement continuer de fonctionner en isolant le composant défaillant. Le passage à l'échelle (scalabilité) est nettement supérieur à celui de l'AMP.

Nouveaux défis : Puisque tous les processeurs exécutent le noyau simultanément, deux cœurs peuvent tenter d'accéder ou de modifier en même temps la même liste de processus ou la même table de mémoire. Sans précaution, la mémoire du noyau est rapidement corrompue.

1.4 Tableau comparatif AMP et SMP

Critère de comparaison	Modèle Asymétrique (AMP)	Modèle Symétrique (SMP)
Exécution du noyau	Réservée au seul processeur maître.	Tous les processeurs peuvent exécuter le noyau.
Complexité du code OS	Faible. Proche d'un OS monoprocesseur.	Élevée. Nécessite une synchronisation stricte.
Goulot d'étranglement	Élevé (le processeur maître sature rapidement).	Réparti (dépend de la finesse des verrous).
Passage à l'échelle	Très limité (stagne après quelques cœurs).	Excellent (extensible à des dizaines/centaines de cœurs).
Résilience aux pannes	Critique (la panne du maître plante l'OS).	Bonne (isolation possible d'un cœur défaillant).

1.5 L'organisation interne du noyau en SMP

Pour éviter la corruption de données en mode SMP, les concepteurs de systèmes d'exploitation ont dû repenser l'organisation interne du noyau. Plusieurs approches ont évolué au fil des décennies :

1. Le verrou global (Big Kernel Lock - BKL) : C'est la méthode historiquement employée pour transformer rapidement un OS mono-cœur en OS compatible SMP (notamment dans les premières versions SMP de Linux ou FreeBSD). Un unique verrou géant protège l'intégralité du noyau. Lorsqu'un processeur souhaite exécuter un appel système, il doit acquérir ce verrou. Si un autre cœur détient déjà le verrou, le second doit attendre.

> **ANALOGIE : LA MAIRIE AVEC UN UNIQUE TAMPON OFFICIEL**
> Imaginez une grande mairie avec dix guichets ouverts. Mais pour valider n'importe quel document administratif, il n'existe qu'un seul tampon dans tout le bâtiment. Un seul employé peut tamponner une feuille à la fois. Même si vous avez dix employés, le rythme de validation reste celui d'une seule personne.

2. Les verrous à gros grain (Coarse-grained locking) : Le noyau est divisé en grands sous-systèmes fonctionnels (gestionnaire de mémoire, pile réseau, système de fichiers, ordonnanceur). Chaque sous-système possède son propre verrou. Le cœur n°1 peut traiter du réseau pendant que le cœur n°2 alloue de la mémoire, mais deux cœurs ne peuvent pas manipuler la pile réseau simultanément.

3. Les verrous fins (Fine-grained locking) : L'approche la plus performante mais la plus complexe. Les verrous ne protègent plus des sous-systèmes entiers, mais des structures de données individuelles (une liste chaînée spécifique, le bloc d'informations d'un seul processus, un nœud d'arbre de mémoire). Le parallélisme est maximal, mais le risque d'erreurs de programmation (comme les blocages mutuels) augmente considérablement.

Exemple historique de Linux : Linux a introduit le *Big Kernel Lock* (connu sous le nom de `lock_kernel()`) dans la version 2.0 en 1996. Il a fallu plus de 15 ans d'efforts communautaires pour découper ce verrou en des milliers de verrous ultra-fins, jusqu'à la suppression définitive du BKL dans la version 2.6.39 publiée en 2011.

Note sur les micro-noyaux : Une autre approche d'organisation réside dans les architectures à micro-noyau (comme Mach ou seL4), où les services traditionnels du noyau (fichiers, réseau) s'exécutent sous forme de serveurs en espace utilisateur, réduisant mécaniquement la surface de code du noyau devant être synchronisée en SMP.

> *Maintenant que nous savons comment le noyau s'organise et se protège en SMP, une question pratique se pose : comment décide-t-on quel processeur doit exécuter quel programme utilisateur à un instant donné ? C'est le rôle de l'ordonnanceur.*

Partie 2 : L'ordonnancement des processus

L'ordonnanceur (scheduler) est l'un des composants les plus critiques d'un système d'exploitation. En monoprocesseur, il se contente de décider dans quel ordre exécuter la file des programmes en attente. En multiprocesseur, le problème devient tridimensionnel : il faut décider *quel* processus exécuter, *quand* l'exécuter, et sur *quel cœur* le placer.

2.1 Structure des files d'attente

La première décision d'architecture concerne l'organisation des tâches prêtes à s'exécuter (état *Ready*) :

La file globale unique (Shared Runqueue) : Une seule liste centrale contient tous les processus en attente. Dès qu'un cœur de processeur se libère, il accède à cette file commune et extrait la tâche prioritaire suivante.

- *Avantage :* Équilibrage parfait et automatique du travail. Aucun processeur ne peut rester inactif s'il existe du travail dans la file.
- *Inconvénients :* La file globale doit être protégée par un verrou. À mesure que le nombre de cœurs augmente, tous les processeurs se battent pour accéder à cette même liste, créant une forte contention. De plus, un processus a de fortes chances d'être exécuté sur un cœur différent à chaque fois qu'il reprend la main, détruisant l'efficacité de la mémoire cache.

Files locales par processeur (Per-CPU Runqueues) : Chaque cœur de processeur gère sa propre file d'attente indépendante.

- *Avantage :* Pas de conflit d'accès entre les cœurs lors de la sélection d'une tâche. Excellente conservation des données en mémoire cache locale.
- *Inconvénient :* Risque important de déséquilibre. Un cœur peut terminer rapidement ses tâches et se retrouver inactif, tandis qu'un cœur voisin reste submergé sous une file de calculs intenses.

> **ANALOGIE : LES CAISSES DU SUPERMARCHÉ**
> La file globale correspond au système de file unique de certaines banques : tout le monde attend dans un seul serpentin et le premier guichet libre prend le client suivant. Les files locales correspondent aux caisses traditionnelles de supermarché : chaque client choisit sa file. Si un client devant vous sort un grand nombre d'articles, vous restez coincé tandis que la caisse voisine avance à toute vitesse.

Le compromis moderne : Tous les OS modernes d'entreprise (notamment l'ordonnanceur EEVDF / CFS de Linux) adoptent des files d'attente locales par processeur, associées à un mécanisme dynamique d'équilibrage de charge pour corriger les asymétries.

2.2 Équilibrage de charge (Load Balancing)

Pour corriger la stase de certains cœurs en présence de files locales, l'OS applique des stratégies de réallocation de tâches :

- **La migration poussée (Push Migration) :** Un sous-système périodique du noyau évalue la charge globale de la machine. S'il repère un déséquilibre marqué entre les files d'attente, il déplace de force ("pousse") des processus depuis les cœurs surchargés vers les cœurs sous-utilisés.
- **La migration tirée (Pull Migration / Work Stealing) :** Lorsqu'un cœur se retrouve totalement inactif car sa file locale est vide, il prend lui-même l'initiative d'examiner les files de ses voisins. S'il trouve un cœur surchargé, il lui "vole" une ou plusieurs tâches en attente (principe du *work stealing*) pour les exécuter immédiatement.

Le coût caché de la migration : Migrer un processus d'un cœur A vers un cœur B n'est jamais gratuit. Les données utiles au programme étaient stockées dans les mémoires caches rapides L1 et L2 du cœur A. Sur le cœur B, ces caches sont "froids". Le processeur doit recharger toutes ses données depuis la mémoire RAM lente, ce qui dégrade temporairement les performances.

2.3 Affinité de processeur (CPU Affinity)

Pour éviter les déplacements inutiles et préserver la réutilisation des mémoires caches, l'ordonnanceur met en œuvre la notion d'**affinité processeur** :

- **Affinité souple (Soft Affinity) :** L'OS essaie de maintenir un processus sur le même cœur autant que possible. Ce n'est qu'en cas de déséquilibre de charge important qu'il acceptera de le migrer vers un autre cœur.
- **Affinité dure (Hard Affinity) :** L'utilisateur ou le développeur spécifie explicitement au système (via des appels système comme `sched_setaffinity()` sous Linux) que son programme n'a le droit de s'exécuter que sur un sous-ensemble strict de cœurs (par exemple uniquement sur les cœurs 0 et 1).

L'affinité et l'équilibrage de charge sont deux forces antagonistes : l'ordonnanceur doit sans cesse naviguer entre le maintien de la localité mémoire et la répartition équitable de la puissance de calcul.

2.4 Ordonnancement sensible à la topologie (NUMA)

Sur les architectures de serveurs modernes possédant plusieurs processeurs physiques, l'accès à la mémoire RAM devient non-uniforme. On parle d'architecture **NUMA (Non-Uniform Memory Access)**.

Dans un système NUMA, la mémoire RAM est découpée et rattachée physiquement à des nœuds (groupes de cœurs). Un cœur accède très rapidement à sa mémoire locale (rattachée à son propre nœud), mais met beaucoup plus de temps à lire la mémoire distante (rattachée à un autre nœud), car l'information doit traverser un bus d'interconnexion (comme AMD Infinity Fabric ou Intel UPI).

> **ANALOGIE : L'ENTREPÔT LOGISTIQUE RÉGIONAL**
> Travailler en mémoire locale NUMA, c'est comme prendre des fournitures dans le placard de votre propre bureau. Aller chercher de la mémoire distante sur un autre nœud NUMA, c'est devoir envoyer un coursier chercher un carton dans un entrepôt situé dans la ville voisine. Le coût en temps est démultiplié.

L'ordonnanceur d'un OS moderne doit donc être "NUMA-aware" :

- Il tente de placer un processus et la mémoire qu'il alloue sur le même nœud NUMA.
- Il évite absolument de migrer un processus vers un autre nœud NUMA, sauf si le déséquilibre de charge devient dramatique.
- Il prend en compte la hiérarchie complète du matériel : cœurs partageant un cache L2, cœurs d'un même socket, puis sockets distants.

> *Maintenant que nous savons comment placer et ordonnancer les processus sur plusieurs cœurs, abordons la difficulté centrale du multiprocessage : comment s'assurer que ces travaux parallèles ne détruisent pas la cohérence des données ?*

Partie 3 : Synchronisation, concurrence et pièges logiciels

Le parallélisme offre un gain de performance impressionnant, mais introduit une complexité redoutable. Dès lors que plusieurs unités de calcul manipulent simultanément le même espace mémoire, des comportements imprévisibles et destructeurs apparaissent si aucun mécanisme de contrôle n'est imposé.

3.1 Le besoin de synchronisation

Une **condition de concurrence (race condition)** survient lorsque le résultat d'une opération dépend de l'ordre d'exécution exact et non déterministe dans lequel les processeurs exécutent leurs instructions.

Considérons l'exemple classique d'une variable globale répertoriant le nombre de clients connectés, initialement égale à 10. Deux cœurs exécutent simultanément l'instruction `compteur++` :

1. Au niveau de l'assembleur, une incrémentation nécessite 3 étapes : lire la valeur en mémoire, lui ajouter 1 dans un registre, puis réécrire le résultat en mémoire.
2. Si le Cœur 1 et le Cœur 2 lisent la mémoire en même temps, ils voient tous les deux la valeur 10.
3. Le Cœur 1 calcule 10 + 1 = 11 et écrit 11.
4. Le Cœur 2 calcule 10 + 1 = 11 et écrit 11.

Résultat final : le compteur vaut 11 alors que deux clients se sont connectés. Une mise à jour a été perdue.

La portion de code qui accède à cette ressource partagée est appelée **section critique**. Elle doit respecter le principe d'**exclusion mutuelle** : un seul processeur à la fois peut s'y trouver.

L'insuffisance de la désactivation des interruptions : Sur un système monoprocesseur antique, il suffisait de désactiver les interruptions logiques avant de rentrer en section critique pour empêcher le système de donner la main à une autre tâche. En multiprocesseur, cette technique est inefficace : désactiver les interruptions sur le cœur n°1 n'empêche absolument pas le cœur n°2 de continuer sa course et de modifier la mémoire en même temps !

La solution matérielle : les instructions atomiques : Pour résoudre ce problème à la racine, les processeurs modernes intègrent des instructions matérielles indivisibles (atomiques), comme `Test-and-Set` ou `Compare-and-Swap (CAS)`. Ces instructions verrouillent le bus mémoire le temps d'exécuter une lecture et une écriture en un seul cycle indivisible, fournissant le socle matériel de tous les verrous logiciels.

3.2 Les outils de synchronisation logicielle

Sur la base des instructions atomiques, le système d'exploitation propose différents mécanismes de verrouillage aux développeurs et à son propre noyau :

1. Le Spinlock (Verrou à attente active) : Lorsqu'un processeur tente d'acquérir un spinlock déjà occupé par un autre cœur, il entre dans une boucle d'attente infinie (*busy waiting*) en testant continuellement l'état du verrou jusqu'à ce qu'il se libère.

- *Avantage :* Temps de réaction ultra-rapide. Il n'y a aucun changement de contexte ni mise en sommeil du processus.
- *Inconvénient :* Consomme 100 % d'un cœur de processeur à tourner dans le vide pendant l'attente. À ne réserver qu'à des sections critiques extrêmement courtes (quelques nanosecondes) au sein du noyau.

2. Le Mutex (Mutual Exclusion Lock) : Si le verrou n'est pas disponible, le processus demandeur est suspendu par le système d'exploitation et placé dans une file d'attente. Le cœur de processeur est immédiatement libéré pour exécuter une autre tâche.

- *Avantage :* Ne gaspille pas de cycles processeur lors d'une longue attente.
- *Inconvénient :* Suspendre un processus et en charger un autre implique un changement de contexte (context switch), une opération lourde en temps de calcul.

3. Le Sémaphore : Imaginé par Edsger Dijkstra, le sémaphore est une variable entière associée à une file d'attente. Il permet de gérer l'accès à un quota fixé de ressources identiques (par exemple limiter à 5 le nombre d'accès simultanés à une base de données).

Verrous hybrides modernes : Pour concilier la vitesse du spinlock et la sobriété du mutex, les noyaux modernes utilisent des verrous adaptatifs : le processeur "tourne" en attente active pendant quelques microsecondes dans l'espoir que le verrou se libère rapidement, puis s'endort en mutex classique si l'attente se prolonge.

3.3 Les pièges et bugs de concurrence

Même avec ces outils, la programmation multiprocesseur est truffée de pièges redoutables :

1. Le Deadlock (Interblocage) : Situation gelée où deux processus restent bloqués à tout jamais, chacun attendant une ressource détenue par l'autre.

> **ANALOGIE : LE CARREFOUR À QUATRE SENS DE PRIORITÉ**
> Quatre voitures arrivent exactement en même temps sur un carrefour sans feu de signalisation, chacune devant céder le passage à sa droite. Chaque conducteur attend que son voisin de droite passe. Tout le monde est bloqué indéfiniment.

Un interblocage ne peut survenir que si les 4 conditions formelles de Coffman sont réunies simultanément : l'exclusion mutuelle, la détention et attente, l'absence de réquisition (non-preemption) et l'attente circulaire. La stratégie classique de prévention consiste à imposer un **ordre strict et global d'acquisition des verrous** dans tout le code.

2. La cohérence de cache (Cache Coherency) : Chaque cœur possède son propre cache L1/L2. Si le Cœur 1 modifie une variable dans son cache local, le Cœur 2 (qui possède une copie ancienne de cette variable dans son propre cache) risque de lire une valeur obsolète. Les puces matérielles intègrent des protocoles de cohérence de cache (comme le protocole **MESI** : Modified, Exclusive, Shared, Invalid) qui invalident automatiquement les copies obsolètes à travers le bus d'interconnexion.

3. Le Faux Partage (False Sharing) : Il s'agit d'un bug de performance particulièrement pernicieux. La mémoire cache ne transfère pas les données octet par octet, mais par blocs de 64 octets appelés *lignes de cache (cache lines)*. Si le Cœur 1 modifie la variable X et que le Cœur 2 modifie la variable Y, et que X et Y ont le malheur de se trouver côte à côte dans la même ligne de cache de 64 octets, le matériel va croire à un conflit. La ligne de cache va faire des allers-retours incessants entre les deux cœurs (effet d'éviction), effondrant les performances alors même que les deux programmes n'ont aucune donnée en commun !

> *À mesure que nous multiplions les verrous pour protéger la mémoire et éviter les erreurs, nous faisons émerger un nouveau problème : les processeurs passent leur temps à s'attendre les uns les autres, limitant le rendement global.*

Partie 4 : Limites théoriques et évolution des systèmes

Ajouter toujours plus de cœurs de processeur sur une puce ne garantit absolument pas une accélération linéaire des calculs. Le logiciel se heurte à des barrières mathématiques et physiques fondamentales.

4.1 La saturation par les verrous (Lock Contention)

La contention de verrou désigne la situation où plusieurs processeurs se disputent simultanément l'accès au même verrou. Lorsque la contention augmente, l'efficacité du système s'effondre : la majorité des cœurs passent leur puissance de calcul non pas à exécuter du code utile, mais à tourner en attente active ou à gérer des changements de contexte.

C'est précisément ce qui s'est produit avec le *Big Kernel Lock*. Pour contourner cet écueil, la recherche sur les OS s'est orientée vers :

- L'utilisation de structures de données propres à chaque processeur (variables *Per-CPU*).
- La conception d'algorithmes dits **Lock-Free** ou **Wait-Free** basés uniquement sur des instructions atomiques sans aucun verrou bloquant.
- L'utilisation de mécanismes de lecture prioritaire comme RCU (Read-Copy-Update) dans Linux, permettant des lectures simultanées sans aucun verrouillage.

4.2 La loi d'Amdahl appliquée au logiciel

Formulée dès 1967 par Gene Amdahl, cette loi mathématique calcule le gain maximal théorique attendu lors du parallélisme d'un programme :

$$\text{Accélération globale (Speedup)} = 1 / [(1 - p) + (p / n)]$$

Où :

- `p` désigne la proportion du programme qui est strictement parallélisable.
- `(1 - p)` représente la fraction séquentielle (incapable d'être exécutée en parallèle, comme les sections critiques sous verrou).
- `n` est le nombre de cœurs de processeur.

Exemple chiffré révélateur : Prenons un logiciel dont 95 % du code est parfaitement parallélisable (`p = 0,95`) et 5 % du code reste strictement séquentiel (`1 - p = 0,05`).

- Avec 16 processeurs : `Speedup = 1 / [ 0,05 + (0,95 / 16) ] = 1 / [ 0,05 + 0,0593 ] ≈ 9,14` . Le gain réels n'est que de 9 fois la vitesse initiale, bien loin des 16 théoriques.
- Même avec un nombre *infini* de processeurs (`n → &infty;`), la limite maximale absolue d'accélération ne dépassera jamais `1 / 0,05 = 20` !

Pour le système d'exploitation, cela signifie que le moindre verrou centralisé ou la moindre phase d'attente dans le noyau augmente la fraction séquentielle `(1 - p)` et réduit à néant le bénéfice des serveurs à 64 ou 128 cœurs.

Nuance avec la loi de Gustafson : La loi d'Amdahl suppose que la taille du problème reste fixe. En 1988, John Gustafson a souligné qu'en pratique, lorsqu'on dispose de plus de puissance, on augmente la taille des données à traiter. La fraction séquentielle devient alors proportionnellement de plus en plus négligeable, permettant un passage à l'échelle bien plus favorable.

4.3 L'évolution vers l'ordonnancement asymétrique (big.LITTLE)

Alors que le modèle SMP repose sur l'hypothèse de cœurs rigoureusement identiques, le retour fracassant du mur de la puissance dans l'informatique mobile et moderne a imposé une nouvelle architecture : l'hétérogénéité matérielle.

Pionnière dans ce domaine, la technologie **ARM big.LITTLE** (déclinée aujourd'hui sous les architectures hybrides Intel Alder Lake et Apple Silicon) associe sur la même puce :

- **Des cœurs à forte performance (P-cores / big) :** Très rapides et gourmands en énergie, parfaits pour les jeux ou le calcul lourd.
- **Des cœurs à haute efficacité énergétique (E-cores / LITTLE) :** Plus lents mais ultra-économes, parfaits pour les tâches de fond.

La tâche de l'ordonnanceur moderne : L'OS ne peut plus traiter les cœurs sur un pied d'égalité symétrique. L'ordonnanceur (comme l'EAS : Energy Aware Scheduling sous Linux/Android) doit continuellement surveiller le comportement des applications. Une tâche d'arrière-plan (synchronisation de mails) est immédiatement reléguée sur un cœur économe, tandis qu'une application de premier plan nécessitant une réactivité immédiate est propulsée sur un cœur de performance.

> *L'histoire boucle ainsi son cycle : le système d'exploitation est parti de l'asymétrie logicielle, a régné avec la symétrie homogène, pour revenir à une gestion asymétrique imposée par la diversité du matériel.*

Conclusion

1. Synthèse globale

L'avènement des systèmes multiprocesseurs n'a pas été le fruit d'un choix logiciel délibéré, mais une réponse impérative aux limites physiques de la matière. La fin du modèle de Dennard et l'apparition simultanée des murs de la puissance, de la fréquence et de la mémoire ont contraint l'industrie à chercher la puissance dans la multiplication des cœurs de calcul.

Face à cette rupture, les systèmes d'exploitation ont dû se métamorphoser à chaque étage de leur architecture :

- **Au niveau du modèle :** Abandon de l'asymétrie maître/esclave au profit du modèle symétrique (SMP) où chaque cœur exécute le noyau.
- **Au niveau de l'organisation :** Évolution douloureuse du verrou global (BKL) vers des milliers de verrous d'une extrême finesse.
- **Au niveau de l'ordonnancement :** Remplacement des files d'attente uniques par des files locales par CPU, régulées par de l'équilibrage de charge, de l'affinité mémoire et la conscience de la topologie NUMA.
- **Au niveau de la concurrence :** Maîtrise des primitives atomiques, des spinlocks et des mutex, tout en contournant les pièges mortels de l'interblocage (deadlock) et du faux partage (false sharing).

L'évolution des systèmes d'exploitation multiprocesseurs illustre une leçon constante en ingénierie informatique : *chaque solution technique crée une nouvelle classe de problèmes, et le rôle de l'OS est d'arbitrer en permanence entre performance, complexité et sécurité.*

2. Ouverture : Les accélérateurs spécialisés (GPU et Puces IA)

Aujourd'hui, l'évolution continue au-delà du processeur généraliste (CPU). L'hétérogénéité s'accentue avec l'intégration massive d'accélérateurs matériels spécialisés : processeurs graphiques (GPU), puces de traitement de signal (DSP) et cartes d'accélération pour l'intelligence artificielle (NPU ou TPU).

Ces puces spécialisées comptent parfois des milliers de cœurs très simples travaillant en parallèle massif (modèle SIMD / SIMT). L'ordonnancement sur ces composants ne s'effectue pas comme sur un CPU traditionnel : l'OS ne gère pas directement chaque fil d'exécution de la puce, mais agit comme un chef d'orchestre de haut niveau à travers des pilotes et des bibliothèques spécialisées (CUDA, ROCm, Vulkan), déléguant le calcul brut à l'accélérateur tout en assurant le transfert sécurisé des données en mémoire.

Le système d'exploitation de demain ne sera plus seulement un gestionnaire de processeurs, mais le régulateur d'un écosystème complexe de puces hétérogènes. Un défi passionnant pour les décennies à venir.